Project Report: Web-Based Email Application

Course: Network Programming (CPAN-226-0NB)

Institution: Humber Polytechnic

Written by: Gurkirat Singh & Ratika

Language/Framework: Python 3.14 / Django 6.0

Table of Contents

1. Abstract
2. Introduction
3. Related Study
4. Application Design
5. Results from the Application
6. Limitations of the Application
7. Conclusion and Further Improvement
8. References

---

a. Abstract

This report presents the design and implementation of a web-based email application built using Python and the Django web framework. The application demonstrates core computer networking concepts — specifically the Simple Mail Transfer Protocol (SMTP) — through a fully functional, browser-accessible interface. Users can register an account, log in, compose and send emails with optional CC recipients and file attachments, and review their sent message history. Email transmission is handled programmatically using Python's built-in `smtplib` library over a TLS-secured SMTP connection to Gmail's mail server. Sent emails are persisted in a local SQLite database. The project bridges theoretical knowledge of client-server communication and network protocols with a practical, real-world implementation.

---

b. Introduction

Email remains one of the most widely used communication protocols on the internet, yet most users interact with it only through polished clients like Gmail or Outlook — with little awareness of the underlying mechanisms. For students studying computer networks, understanding how email is structured, authenticated, and transmitted at the protocol level is essential.

This project addresses that gap by building a web-based email client from scratch. Rather than relying on third-party email libraries that abstract away the details, the application uses Python's standard library (`smtplib`, `email.mime`) to construct and transmit MIME-formatted email messages directly over SMTP. The Django web framework provides the web

server, user authentication, database access, and templating layer.

The core learning objectives of this project are:

- To understand the SMTP protocol and how a mail client interacts with a mail server.
- To implement user authentication and session management in a web application.
- To handle multi-part MIME messages including plain-text bodies and binary file attachments.
- To demonstrate data persistence by logging sent emails to a relational database.

The result is a working email application that any registered user can use through a web browser to send real emails to real recipients.

---

c. Related Study

Several technical documents, standards, and official references were studied during the development of this project.

SMTP Protocol (RFC 5321)

The foundation of this project is the Simple Mail Transfer Protocol, formally defined in **RFC 5321** [1]. This document specifies how mail transfer agents exchange email messages, including the command/response sequences (`EHLO`, `AUTH`, `MAIL FROM`, `RCPT TO`, `DATA`), connection establishment, and error codes. Understanding this RFC clarified how `smtplib` operates under the hood and why `STARTTLS` is issued before authentication.

MIME Standard (RFC 2045–2049)

Email messages that carry formatted content or attachments must conform to the **Multipurpose Internet Mail Extensions (MIME)** specification [2]. RFC 2045 defines the structure of MIME messages, including `Content-Type` and `Content-Transfer-Encoding` headers. This was essential for constructing `MIMEMultipart`, `MIMEText`, and `MIMEBase` objects when attaching files to outgoing emails.

Python `smtplib` Documentation

The official Python documentation for the `smtplib` module [3] was referenced extensively to understand how to establish a connection to an SMTP server, upgrade it to TLS using `SMTP.starttls()`, authenticate with `SMTP.login()`, and dispatch messages with `SMTP.sendmail()`.

Python `email.mime` Documentation

The `email.mime` package documentation [4] was used to understand how to build multi-part MIME messages programmatically — attaching plain text bodies with `MIMEText` and encoding binary file data with `MIMEBase` using Base64 encoding via `email.encoders`.

Django Official Documentation

The Django documentation [5] was the primary reference for building the web application layer, including:

- The ORM and model definitions (`django.db.models`)
- Built-in user authentication (`django.contrib.auth`)
- Form handling and validation (`django.forms`)
- File upload handling and `MEDIA_ROOT` configuration
- URL routing and the request/response cycle

TLS/STARTTLS and Gmail SMTP

Google's support documentation [6] was referenced to understand Gmail's SMTP server requirements: host `smtp.gmail.com`, port `587`, TLS via STARTTLS, and the requirement to use an **App Password** rather than the account password when 2-Step Verification is enabled.

---

d. Application Design

Overview

The application follows Django's **MVT (Model-View-Template)** architectural pattern. The `mailer` Django app handles all email-related functionality, while the `emailapp` package contains project-wide configuration.

Python Version

- **Python 3.14.3**
- **Django 6.0.3**

Dependencies

Package	Version	Purpose
`django`	6.0.3	Web framework (routing, ORM, auth, templates)
`python-dotenv`	1.2.2	Load SMTP credentials from `.env` file
`smtplib`	stdlib	SMTP connection and message delivery
`email.mime`	stdlib	Construct MIME-formatted email messages
`sqlite3`	stdlib	Database backend (via Django ORM)

No external email-sending libraries (e.g., `sendgrid`, `yagmail`) were used. All SMTP logic is implemented using Python's standard library to demonstrate the protocol directly.

Data Storage

Django's default **SQLite** database (`db.sqlite3`) is used. Two tables are relevant:

- **`auth_user`** — Django's built-in user table. Stores username, hashed password, and email.
- **`mailer_sentemail`** — Custom model that logs every sent email:

```
class SentEmail(models.Model):
    sender     = models.ForeignKey(User, on_delete=models.CASCADE)
    to         = models.TextField()
    cc         = models.TextField(blank=True)
```

```
    subject    = models.CharField(max_length=255)
    body       = models.TextField()
    attachment = models.FileField(upload_to='attachments/', blank=True,
    sent_at    = models.DateTimeField(auto_now_add=True)
```

File attachments are stored on disk under the `media/attachments/` directory and referenced in the database by file path.

SMTP Email Flow

Email transmission follows these steps, implemented in `mailer/views.py`:

1. A `MIMEMultipart` object is created and headers (`From`, `To`, `CC`, `Subject`) are set.
2. The email body is attached as a `MIMEText('plain')` part.
3. If a file was uploaded, it is read, Base64-encoded as a `MIMEBase` object, and attached.
4. An SMTP connection is opened to `smtp.gmail.com:587`.
5. `EHLO` is issued, then `STARTTLS` to upgrade to a TLS-encrypted connection.
6. The client authenticates with `LOGIN` using credentials from `.env`.
7. The message is dispatched via `sendmail()` to all recipients (To + CC).

```
with smtplib.SMTP('smtp.gmail.com', 587) as server:
    server.ehlo()
    server.starttls()
    server.login(EMAIL_HOST_USER, EMAIL_HOST_PASSWORD)
    server.sendmail(EMAIL_HOST_USER, all_recipients, msg.as_string())
```

User Authentication

Django's built-in authentication system (`django.contrib.auth`) is used. Passwords are stored as salted hashes (PBKDF2-SHA256 by default). The `@login_required` decorator protects the Compose, Sent, and Detail views — unauthenticated requests are redirected to `/login/`.

URL Structure

URL	View	Description
`/login/`	`login_view`	Authenticate existing user
`/register/`	`register_view`	Create new account
`/compose/`	`compose_view`	Compose and send email
`/sent/`	`sent_view`	List of sent emails
`/sent//`	`email_detail_view`	Full details of one email
`/logout/`	`logout_view`	End session

Configuration

Sensitive credentials are stored in a `.env` file (excluded from version control via `.gitignore`) and loaded at startup using `python-dotenv`:

```
EMAIL_HOST_USER=you@gmail.com
EMAIL_HOST_PASSWORD=your_app_password
```

Running the Application

```
# 1. Activate virtual environment
source venv/bin/activate

# 2. Install dependencies
pip install django python-dotenv

# 3. Apply database migrations
python manage.py migrate

# 4. Start the development server
python manage.py runserver
```

The application is then accessible at `http://127.0.0.1:8000`.

---

e. Results from the Application

The application was tested end-to-end: a user account was created, an email with a PDF attachment was sent to a real Gmail address, and the message was received successfully.

Registration Page (`/register/`)

The registration page presents a form for username, email address, and password (with confirmation). Django's `UserCreationForm` enforces minimum password length and prevents duplicate usernames. On success, the user is automatically logged in and redirected to Compose.

```
+-----------------------------------------------+
|  ✉ WebMail — Create Account                   |
|-----------------------------------------------|
|  Username:   [________________]               |
|  Email:      [________________]               |
|  Password:   [________________]               |
|  Confirm:    [________________]               |
|                                               |
|         [ Create Account ]                    |
|  Already have an account? Login               |
+-----------------------------------------------+
```

Login Page (`/login/`)

Returns an error message for invalid credentials. Valid credentials establish a session and redirect to `/compose/`.

Compose Page (`/compose/`)

The core interface of the application. Accepts To (multiple comma-separated addresses), CC, Subject, Body, and an optional file attachment. On submission, the email is sent via SMTP and the user is redirected to the Sent folder. A green success banner or red error message is shown depending on the outcome.

```
+-----------------------------------------------+
|  Compose Email                                |
|-----------------------------------------------|
|  To:          [_______________]               |
|  CC:          [_______________]               |
|  Subject:     [_______________]               |
|  Message:     [                           ]   |
|               [                           ]   |
|               [                           ]   |
|  Attachment: [  Choose File  ]                |
|                                               |
|          [ ▶ Send Email ]                     |
+-----------------------------------------------+
```

Sent Folder (`/sent/`)

Lists all emails sent by the currently logged-in user, sorted by most recent first. Each row shows the subject line, recipient address, and timestamp. Clicking a row opens the detail view.

```
+-----------------------------------------------+
|  Sent Emails                                  |
|-----------------------------------------------|
|  Project Submission      To: prof@humber.ca   |
|                          Apr 5, 2026 14:32    |
|-----------------------------------------------|
|  Hello from WebMail      To: test@gmail.com   |
|                          Apr 5, 2026 13:10    |
+-----------------------------------------------+
```

Email Detail View (`/sent/<id>/`)

Shows the full details of a sent email: From, To, CC, timestamp, full message body, and a download link for any attached file.

SMTP Error Handling

If the credentials in `.env` are incorrect, the application catches `SMTPAuthenticationError` and displays a descriptive error message in the UI rather than crashing.

---

f. Limitations of the Application

1. **No inbox / receiving emails.** The application is a send-only client. It does not implement IMAP or POP3, so incoming messages cannot be viewed. Receiving email requires a different protocol and is significantly more complex.
2. **Single sender identity.** All emails are sent from the single Gmail account configured in `.env`. Users log into the web app but do not have individual email addresses — the sender address is always the configured Gmail account.
3. **No HTML email support.** The email body is sent as `text/plain`. HTML-formatted emails with styling, hyperlinks, or embedded images are not supported.

4. **Gmail-specific configuration.** The SMTP settings are hardcoded for Gmail (`smtp.gmail.com`, port 587). Supporting other providers (Outlook, Yahoo, custom SMTP) would require configuration changes.
5. **App Password dependency.** Gmail requires an App Password when 2-Step Verification is enabled, adding setup friction. This is a Gmail security policy, not a flaw in the application.
6. **No pagination.** The Sent folder loads all emails at once. For a user with hundreds of sent emails, this would degrade performance.
7. **No email search or filtering.** There is no way to search the sent history by recipient, subject, or date range.
8. **Development server only.** The app runs on Django's built-in development server, which is not suitable for production use. A production deployment would require a WSGI server (e.g., Gunicorn) and a reverse proxy (e.g., Nginx).
9. **Single-user SMTP credentials.** Credentials are stored in a `.env` file on the server. A production system would use per-user OAuth 2.0 tokens (Gmail API) rather than a shared App Password.

---

g. Conclusion and Further Improvement

Conclusion

This project successfully demonstrates the practical application of the SMTP protocol within a full-stack web application. By building the email client using Python's standard library rather than high-level abstractions, the project provides direct insight into how email messages are structured as MIME documents, how a client authenticates with a mail server, and how TLS secures the connection. The Django framework handled the web layer cleanly, allowing focus to remain on the networking logic. All specified features — user authentication, email composition with CC and attachments, SMTP delivery, and a database-backed sent folder — were implemented and verified with real email delivery.

Further Improvements

The following enhancements would improve the application in future iterations:

- **Inbox via IMAP:** Integrate Python's `imaplib` to fetch and display incoming emails, making it a complete two-way client.
- **Per-user OAuth 2.0:** Replace the shared App Password with Gmail's OAuth 2.0 flow so each user sends from their own Gmail account.
- **HTML email composition:** Add a rich-text editor (e.g., TinyMCE) to support HTML-formatted email bodies.
- **Multi-provider SMTP:** Allow users to configure their own SMTP server settings (host, port, credentials) to support non-Gmail accounts.
- **Search and pagination:** Add full-text search over the sent folder and paginate results for scalability.
- **Email scheduling:** Allow users to schedule emails to be sent at a future date/time using Django's task queue (e.g., Celery with Redis).
- **Production deployment:** Package the application with Docker, serve it via Gunicorn + Nginx, and use PostgreSQL instead of SQLite.
- **Draft saving:** Auto-save compose form content as a draft so work is not lost if the browser is closed.

References

[1] J. Klensin, "Simple Mail Transfer Protocol," RFC 5321, Internet Engineering Task Force (IETF), October 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5321

[2] N. Freed and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies," RFC 2045, IETF, November 1996. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2045

[3] Python Software Foundation, "`smtplib` — SMTP protocol client," Python 3.14 Documentation. [Online]. Available: https://docs.python.org/3/library/smtplib.html

[4] Python Software Foundation, "`email.mime` — Creating email and MIME objects from scratch," Python 3.14 Documentation. [Online]. Available: https://docs.python.org/3/library/email.mime.html

[5] Django Software Foundation, "Django Documentation," Django 6.0. [Online]. Available: https://docs.djangoproject.com/en/6.0/

[6] Google LLC, "Send email with Gmail SMTP," Google Workspace Admin Help. [Online]. Available: https://support.google.com/a/answer/176600

[7] J. Postel, "Internet Standard: Simple Mail Transfer Protocol," RFC 821, IETF, August 1982 (original SMTP specification). [Online]. Available: https://www.rfc-editor.org/rfc/rfc821

[8] P. Resnick (Ed.), "Internet Message Format," RFC 5322, IETF, October 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5322